

3.1 - GitHub Actions: Kubernetes

Introduction

GitHub Actions also have the ability to execute commands on our local host machine, as we have explored in the previous assignment.

We are going to use this functionality to work with Kubernetes commands in order to update production-style deployments in a professional way.

Objective

Setting up GitHub Actions to deploy applications in a Docker Container in Kubernetes Pods in a scalable Deployment, with Services configured.

Extra: Implementing Helm to deploy our application all in one.

Knowledge

- GitHub Actions
- Kubernetes Deployments & Services
- Helm Charts

Skills

- Deploying Kubernetes applications on a local cluster using GitHub Actions

Necessities

- A Kubernetes Cluster as set up in
 - Docker Desktop
 - Installed `kubect` (and accessible in your path)
 - Installed `k3d` (and accessible in your path)
 - Installed `helm` (and accessible in your path)
- A Local Runner as set up in

- A terminal
- A Git Repository on GitHub
- An IDE such as Visual Studio Code

Introduction

Objective

Knowledge

Skills

Necessities

GitHub Actions + Kubectl

First trials

Deployments

Getting Helm involved

Complete automation

A few notes

What did you learn?

Give three interesting exam questions

Handing in this assignment

GitHub Actions + Kubectl

First trials

GitHub Actions has a way of connecting to your local Kubernetes instance by using `kubectl`, the same interface you are using yourself and using the self-hosted GitHub Actions runner.

To test out if your local runner can access your Kubernetes cluster, you can try to install a Kubernetes object with the `kubectl` commands.

👉 If it does not work, try and give the path to your `~/.kube/config` file as a flag in all the `kubectl` commands.

1. Create a new workflow
2. Trigger it manually `workflow_dispatch` or through a `push` on the main-branch

3. Make sure it uses the `self-hosted` runner this time.
4. Add the right actions:
 - The action to pull your source code
 - A `kubectl` action which will apply a Kubernetes object to your cluster
 - A `kubectl` action to check for the pods in your namespace
5. Find a way to generate a random name for the Kubernetes Pod object.
You can find an example file in here:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod-{ID} # Try to make this dynamically generated (random)
  namespace: <your-namespace> # Enter your namespace here
labels:
  name: mypod
spec:
  containers:
  - name: mypod
    image: rancher/hello-world
```



QUESTION 1 - Random name

How did you make the name randomised?



ANSWER 1 - Random name

GitHub Actions snippet

env:

```
RAND_SUFFIX: ${GITHUB_RUN_ID}-${GITHUB_RUN_ATTEMPT}
```

steps:

- name: Generate resource names

run: |

```
echo "APP_NAME=nginx-${RAND_SUFFIX}" >> $GITHUB_ENV
```



QUESTION 2 - Screenshot & logs

Show the result of the pod running in the terminal.



ANSWER 2 - Screenshot & logs

kubectl get pods -o wide

kubectl logs deploy/nginx-deployment --tail=20

```
PS C:\School\VL0Ps\week3\ga-hello-nginx> kubectl get pods -o wide
>> kubectl logs deploy/nginx-deployment --tail=20
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-deployment-78cbc5954f-54gt9	1/1	Running	0	14m	10.1.0.6	docker-desktop	<none>	<none>

```

2025/11/05 18:40:24 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2025/11/05 18:40:24 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2025/11/05 18:40:24 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/11/05 18:40:24 [notice] 1#1: start worker processes
2025/11/05 18:40:24 [notice] 1#1: start worker process 30
2025/11/05 18:40:24 [notice] 1#1: start worker process 31
2025/11/05 18:40:24 [notice] 1#1: start worker process 32
2025/11/05 18:40:24 [notice] 1#1: start worker process 33
2025/11/05 18:40:24 [notice] 1#1: start worker process 34
2025/11/05 18:40:24 [notice] 1#1: start worker process 35
2025/11/05 18:40:24 [notice] 1#1: start worker process 36
2025/11/05 18:40:24 [notice] 1#1: start worker process 37
2025/11/05 18:40:24 [notice] 1#1: start worker process 38
2025/11/05 18:40:24 [notice] 1#1: start worker process 39
2025/11/05 18:40:24 [notice] 1#1: start worker process 40
2025/11/05 18:40:24 [notice] 1#1: start worker process 41
192.168.65.3 - - [05/Nov/2025:18:40:37 +0000] "GET / HTTP/1.1" 200 278 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
2025/11/05 18:40:38 [error] 30#30: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 192.168.65.3, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "localhost:30080", referer: "http://localhost:30080/"
192.168.65.3 - - [05/Nov/2025:18:40:38 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:30080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
192.168.65.3 - - [05/Nov/2025:18:40:39 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36" "-"
PS C:\School\VL0Ps\week3\ga-hello-nginx>

```

Deployments

Similar to how a Pod can be deployed in `kubectl`, we can also use a Deployment. Set this up by creating a deployment and service with the necessary settings, such as NodePort to open up the application to the host machine.

However, when using a deployment, we are usually having a new Docker image to deploy. This image can be automated in the pipeline as we will do later. But when you're choosing to use the same image name every time, you'll need to restart your deployment so it gets the new image update.

Search for the command to rollout a new update using the same image.



QUESTION 3 - Deployment rollout

Which command did you use for the deployment rollout?



ANSWER 3 - Deployment rollout

```
# GitHub Actions snippet
- name: Rollout deployment
  run: |
    kubectl rollout restart deployment/nginx-deployment
    kubectl rollout status deployment/nginx-deployment --timeout=120s
```

Getting Helm involved

As we know, Helm is great to do automations of the Kubernetes installations. We simply need to enter a command like `helm upgrade --install ...` with all the necessary parameters to install a chart into our cluster.

The useful thing to combine with GitHub Actions is that we can override the `values.yaml` file from the command line as well.

We can even use a special Action / Task that is specialised in editing YAML files. That way we can easily specify a new version of the Docker image.

Try to install a Helm chart into your Kubernetes cluster, with the right GitHub Actions that you can confirm. You can use the same logic as you used above for the Kubernetes part.



QUESTION 4 - Helm install

How does the command for a Helm install look like with the flags added ?



ANSWER 4 - Helm install

```
helm install "$APP_NAME" ./chart \
  --namespace demo --create-namespace \
  --set image.repository=ghcr.io/rojkatomas/hello-world-nginx \
  --set image.tag=v1 \
  --wait --timeout 2m --atomic
```

Complete automation

This quick checklist / manual can help you in figuring out a good structure for the GitHub Actions.

1. Clone the GitHub repository
2. Optional: Perform some automated checks on the code if needed (Unit Test, Integration Test, Cyclomatic Complexity, Code Coverage ...)
3. Build the Docker Image with a new tag (`:latest` , `:<git-sha>`). We like to refer to the Git-SHA as it's always unique and refers back to the Git commit we performed.
4. Push the Docker Image (to GHCR or Docker Hub).
5. Deploy using plain Kubernetes (without Helm) or with Helm

Kubernetes without Helm

Kubernetes with Helm

1. Create a Kubernetes namespace, if needed
2. Change the Kubernetes YAML files
3. Apply the Kubernetes YAML files
4. Test the final result

1. Create a Kubernetes namespace, if needed
2. Override the `values.yaml` file using a command line interface flag or with a specialised task.
3. Upgrade (or install) the Helm chart
4. Test the final result

A few notes

A few things to note using the automation with GitHub Actions:

- Make sure it works when you have just installed a clean Kubernetes cluster:
Cold start
 - ✓ ~~Create namespace if doesn't exist~~
 - ✓ ~~Change properties if needed, or use default properties~~
 - ✓ ~~Update OR install. Don't forget that your `:latest` tag will not update the Kubernetes cluster, as it will only have an effect if the Kubernetes' **Desired State** is different than it was before. That's why we prefer to use **Semantic Versioning** for our Docker Image tags. If you want to overcome that issue, you can use the `ImagePullPolicy: Always` property in your Kubernetes deployment container specs.~~
- Don't have any secrets into your GitHub repository
- See if your GitHub Actions runner is active
- Try automated tests in your project

What did you learn?

Fill in something that you learned during this lesson

|

I learned how to deploy a Docker image to Kubernetes locally using a Deployment

Give three interesting exam questions

1. What is the purpose of a Deployment in Kubernetes?
2. What does a Service of type NodePort do?
3. Which command is used to apply Kubernetes configuration files?

Handing in this assignment

You can hand this in by duplicating this document on Notion, print this document as a `.pdf` and submit that document on Leho.

Also hand in the written Source Code in a `.zip` file please.

Checkboxes:

- ☒ ~~I have duplicated this file~~
- ☒ ~~I have filled in all the answers~~
- ☒ ~~I have added something that I learned~~
- ☒ ~~I have added three interesting exam questions~~
- ☒ ~~I have zipped my project and uploaded it to Leho~~